Roteiro 1 — Pipeline HTTP, binding e Minimal APIs versus MVC

Disciplina: Desenvolvimento Web com .NET — UNIP. **Fonte:** Roteiros.pdf, páginas 3–5. **Consolidação:** 29/10/2026. Dados do aluno, RA, turma e professor devem ser preenchidos pelo responsável pela entrega; não foram presumidos.

Objetivo e introdução

Construir a base executável do Portal de Tarefas e observar o percurso HTTP, utilizando quatro fontes de binding e dois endpoints diagnósticos equivalentes. O relatório descreve a solução cumulativa existente e corrigida, sem reconstruir artificialmente uma execução histórica anterior ao Identity ou ao EF Core.

Ferramentas e estrutura

O projeto `PortalTarefas.Web` tem alvo `net10.0`, SDK selecionado em `global.json`, controllers MVC, Minimal APIs e configuração por ambiente. A versão efetivamente executada e os resultados de build/testes ficam no status de execução. O código completo exigido pelo roteiro está no anexo 01, que deve corresponder à versão final dos arquivos.

Desenvolvimento e funcionalidades

`Program.cs` registra MVC, tratamento de erros, CORS nomeado `FrontendLocal`, documentação OpenAPI e os demais serviços cumulativos. `RequisicaoMiddleware` envolve a requisição, valida o identificador de correlação recebido, devolve `X-Correlation-ID` e registra método, caminho, status e duração sem copiar corpos, cookies, tokens ou senhas.

Ordem do pipeline da solução cumulativa:

1. Registro/correlação de requisição, que envolve a execução restante e registra a saída.
2. Tratamento centralizado de exceções e padronização de respostas sem corpo.
3. HSTS no ambiente apropriado e redirecionamento HTTPS.
4. Arquivos estáticos e seleção de rota.
5. CORS com origem explícita.
6. Autenticação e autorização, incorporadas pelo roteiro 6.
7. Limitação de requisições e output cache, incorporados pelo roteiro 7.
8. Endpoints de diagnóstico, API, MVC, saúde e SPA.

O filtro de ação observa a execução interna do controller. A entrada registrada pelo middleware antecede a ação; sua saída ocorre depois do retorno da ação e do tratamento aplicável. O arquivo final `Program.cs` é a referência para a ordem exata; os logs reais devem corroborá-la.

Fonte de binding	Demonstração
Rota	`{id:int}` nos diagnósticos e na consulta de tarefa
Query string	`filtro`, `pagina`, `tamanho` e filtros de tarefa
Cabeçalho	`X-Cliente` no diagnóstico; `X-Correlation-ID` no middleware
JSON	Corpo de cadastro de tarefa em `POST /api/tarefas`

`GET /api/minimal/diagnostico/{id}` e `GET /api/diagnostico/{id}` devolvem `DiagnosticoDto` com o mesmo conteúdo para os mesmos dados de rota/query/header. As operações de tarefas usam DTOs separados das entidades. Na versão cumulativa, cadastrar exige autenticação e CSRF; os controles posteriores foram preservados ao validar as funcionalidades iniciais.

Decisões técnicas

Eixo	Minimal API	Controller MVC/API
Organização	Delegate próximo ao mapeamento; adequado ao diagnóstico curto	Ações agrupadas por recurso e convenções claras
Dependências	Parâmetros do handler resolvidos pelo contêiner	Injeção no construtor e dependências compartilhadas entre ações
Validação	Requer configuração/comportamento explícito para o modelo	`ApiController`/ModelState e atributos integram a validação
Filtros	Filtros de endpoint têm ciclo próprio	Filtros MVC observam ações e resultados; Razor integra formulários
Testes	Integração HTTP exercita binding e pipeline	Integração HTTP e testes de ação/validação exercitam contratos e Razor

Controllers são priorizados para o CRUD e a interface Razor porque compartilham validação, filtros, DTOs e organização das ações. Minimal APIs permanecem para diagnósticos, saúde e leitura pública curta. Essa escolha atende à combinação de API e MVC exigida nas próximas atividades.

Testes, resultados e evidências

`ApiTests` e os testes básicos do pipeline verificam diagnósticos equivalentes, binding, correlação, redirecionamento e operações HTTP. A execução prioritária consolidada aprovou 4 testes dos roteiros 1 e 2, incluindo o View Component, com transcrições de requests/responses e logs em `docs/evidencias/roteiros-1-2.trx`. As execuções posteriores da suíte cumulativa e os smokes HTTPS estão consolidados em STATUS_DE_EXECUCAO.md; resultados não são inferidos da presença dos testes.

Para reproduzir, seguir o README, compilar a solução, iniciar HTTPS e executar as integrações em banco exclusivo quando o cenário usar SQL/Identity. Exemplos de requests ficam em requisicoes.http. Esse arquivo é procedimento; só uma resposta efetivamente coletada conta como evidência de execução.

Aprimoramento e conclusão

O middleware é reutilizável por classe e extensão de `IApplicationBuilder`, com correlação devolvida ao cliente. Os testes de integração complementam o comportamento observável do pipeline. A implementação está identificada, mas a conclusão da atividade depende das verificações e evidências individuais registradas na matriz; pendências documentais ou de execução permanecem explícitas.

Anexo 01 — Código completo do roteiro 1 na solução cumulativa

Snapshot documental da versão corrigida, consolidado em 29/10/2026. Os recursos dos roteiros posteriores permanecem no Program.cs. O histórico real está no Git; este anexo não simula código de uma entrega anterior.

src/PortalTarefas.Web/Program.cs

```
using System.Net;
using System.Text.Json.Nodes;
using System.Threading.RateLimiting;
using System.Text.Json.Serialization;
using Microsoft.AspNetCore.Diagnostics.HealthChecks;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.RateLimiting;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Http.Resilience;
using Microsoft.OpenApi;
using OpenTelemetry.Metrics;
using OpenTelemetry.Resources;
using OpenTelemetry.Trace;
using PortalTarefas.Web.Controllers;
using PortalTarefas.Web.Data;
using PortalTarefas.Web.DTOs;
using PortalTarefas.Web.Filters;
using PortalTarefas.Web.Infrastructure;
using PortalTarefas.Web.Middleware;
using PortalTarefas.Web.Models;
using PortalTarefas.Web.Security;
using PortalTarefas.Web.Services;

var builder = WebApplication.CreateBuilder(args);
builder.Logging.ClearProviders();
builder.Logging.AddJsonConsole(options => options.IncludeScopes = true);
builder.Services.AddOptions<PortalOptions>().BindConfiguration("Portal").ValidateDataAnnotations()
    .Validate(o => Uri.TryCreate(o.FrontendOrigin, UriKind.Absolute, out var uri) &&
        (uri.Scheme == "https" || uri.Scheme == "http") && uri.AbsolutePath == "/", "Portal:FrontendOrigin deve conter
uma origem HTTP válida.")
    .ValidateOnStart();
// O contexto guarda apenas opções; nenhum estado de usuário/equipe persiste no pool.
// Resolve a configuração final do host, incluindo fontes adicionadas pelo ambiente de testes.
builder.Services.AddDbContextPool<PortalTarefasDbContext>((services, options) =>
{
    var config = services.GetRequiredService<IConfiguration>();
    options.UseSqlServer(config.GetConnectionString("PortalTarefas") ??
        "Server=localhost;Database=PortalTarefasDemo;Integrated Security=true;Encrypt=true;TrustServerCertificate=true");
});
builder.Services.AddSingleton<InMemoryTarefaService>();
builder.Services.AddScoped<EfTarefaService>();
builder.Services.AddScoped<ITarefaService>(services => services.GetRequiredService<IConfiguration>()["Portal:Storage"] ==
"Memory"
    ? services.GetRequiredService<InMemoryTarefaService>() : services.GetRequiredService<EfTarefaService>());

builder.Services.AddIdentity<Usuario, IdentityRole>(o =>
{
    o.Password.RequiredLength = 12;
```

```
    o.Password.RequireDigit = true; o.Password.RequireUppercase = true; o.Password.RequireLowercase = true;
    o.Password.RequireNonAlphanumeric = true;
    o.Lockout.MaxFailedAccessAttempts = 5; o.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(5);
    o.User.RequireUniqueEmail = true;
}).AddEntityFrameworkStores<PortalTarefasDbContext>().AddDefaultTokenProviders();
builder.Services.AddScoped<IUserClaimsPrincipalFactory<Usuario>, UsuarioClaimsFactory>();
builder.Services.ConfigureApplicationCookie(o =>
{
    o.Cookie.Name = "__Host-PortalTarefas"; o.Cookie.HttpOnly = true;
    o.Cookie.SecurePolicy = CookieSecurePolicy.Always; o.Cookie.SameSite = SameSiteMode.Lax;
    o.ExpireTimeSpan = TimeSpan.FromMinutes(30); o.SlidingExpiration = true;
    o.LoginPath = "/Conta/Login"; o.AccessDeniedPath = "/Conta/AcessoNegado";
    o.Events.OnRedirectToLogin = context =>
    {
        if (context.Request.Path.StartsWithSegments("/api")) context.Response.StatusCode = 401;
        else context.Response.Redirect(context.RedirectUri);
        return Task.CompletedTask;
    };
    o.Events.OnRedirectToAccessDenied = context =>
    {
        if (context.Request.Path.StartsWithSegments("/api")) context.Response.StatusCode = 403;
        else context.Response.Redirect(context.RedirectUri);
        return Task.CompletedTask;
    };
});
builder.Services.AddAuthorization(o =>
{
    o.AddPolicy("CriarTarefa", p => p.RequireAuthenticatedUser().RequireRole("Usuario", "Administrador"));
    o.AddPolicy("EditarOutraEquipe", p => p.RequireClaim("tarefas:editar-outra-equipe", "true"));
});
builder.Services.AddAntiforgery(o =>
{
    o.HeaderName = "X-CSRF-TOKEN"; o.Cookie.Name = "__Host-PortalTarefas-CSRF";
    o.Cookie.SecurePolicy = CookieSecurePolicy.Always; o.Cookie.SameSite = SameSiteMode.Lax;
});
builder.Services.AddControllersWithViews(o => { o.Filters.Add<LogAcaoFilter>(); o.Filters.Add<ApiCsrfFilter>(); })
    .AddJsonOptions(o => o.JsonSerializerOptions.NumberHandling = JsonNumberHandling.Strict);
builder.Services.ConfigureHttpJsonOptions(o => o.SerializerOptions.NumberHandling = JsonNumberHandling.Strict);
builder.Services.Configure<ApiBehaviorOptions>(o => o.InvalidModelStateResponseFactory = context =>
{
    var validation = new ValidationProblemDetails(context.ModelState);
    var details = new ApiProblemDetails { Status = 400, Title = "Confira os campos informados.", Errors =
validation.Errors };
    details.Extensions["correlationId"] = context.HttpContext.TraceIdentifier;
    return new BadRequestObjectResult(details) { ContentTypes = { "application/problem+json" } };
});
builder.Services.AddProblemDetails(o => o.CustomizeProblemDetails = c => c.ProblemDetails.Extensions["correlationId"] =
c.HttpContext.TraceIdentifier);
builder.Services.AddExceptionHandler<ErroHandler>();
builder.Services.AddOpenApi(options => options.AddDocumentTransformer((document, _, _) =>
{
    // Contrato reproduzível em desenvolvimento e CI, independente da porta de exportação.
    document.Servers = [new() { Url = "/" }];

    // O esquema permite informar o token no Authorize do Swagger sem duplicar
    // nos DTOs a proteção que o transporte da SPA já acrescenta às requisições.
    document.Components ??= new();
    document.Components.SecuritySchemes ??= new Dictionary<string, IOpenApiSecurityScheme>();
    document.Components.SecuritySchemes["CsrfToken"] = new OpenApiSecurityScheme
    {
        Type = SecuritySchemeType.ApiKey,
        In = ParameterLocation.Header,
        Name = "X-CSRF-TOKEN",
        Description = "Obtenha token em GET /api/auth/csrf e informe seu valor em Authorize. " +
            "Obtenha outro token após login ou logout. O navegador envia o cookie HttpOnly na mesma origem; " +
            "o token CSRF não substitui a autenticação."
    };
    if (document.Paths is { } paths)
    {
```

```
        foreach (var (path, item) in paths)
        {
            if (!path.StartsWith("/api/", StringComparison.Ordinal) || item.Operations is not { } operations) continue;
            foreach (var (method, operation) in operations)
            {
                if (method != HttpMethod.Post && method != HttpMethod.Put && method != HttpMethod.Delete) continue;
                operation.Security = [new OpenApiSecurityRequirement
                {
                    [new OpenApiSecuritySchemeReference("CsrfToken", document)] = []
                }];
            }
        }

        if (paths.TryGetValue("/api/tarefas", out var tarefas) && tarefas.Operations is { } taskOperations &&
            taskOperations.TryGetValue(HttpMethod.Post, out var create))
        {
            // Valores didáticos de contrato; não representam uma execução ou evidência real.
            create.Description = "Exemplos ilustrativos. Informe a equipe da sessão autenticada e o token CSRF obtido
após o login.";
            if (create.RequestBody?.Content is { } requestContent && requestContent.TryGetValue("application/json", out
var requestJson))
                requestJson.Example = JsonNode.Parse("""
                    {"titulo":"Revisar relatório do Portal de Tarefas","descricao":"Conferir testes e evidências antes da
entrega.","situacao":"Pendente","prioridade":2,"prazo":"2026-11-30","equipe":"Geral","publica":true}
                    """);
            if (create.Responses is { } responses)
            {
                if (responses.TryGetValue("201", out var created) && created.Content is { } createdContent &&
                    createdContent.TryGetValue("application/json", out var createdJson))
                    createdJson.Example = JsonNode.Parse("""
                        {"id":1,"titulo":"Revisar relatório do Portal de Tarefas","descricao":"Conferir testes e
evidências antes da entrega.","situacao":"Pendente","prioridade":2,"prazo":"2026-11-
30","equipe":"Geral","publica":true,"proprietarioId":"usuario-exemplo","rowVersion":"AAAAAAAAAAE=","criadaEm":"2026-10-
29T12:00:00+00:00"}
                        """);
                if (responses.TryGetValue("400", out var invalid) && invalid.Content is { } invalidContent &&
                    invalidContent.TryGetValue("application/problem+json", out var invalidJson))
                    invalidJson.Example = JsonNode.Parse("""
                        {"status":400,"title":"Confira os campos informados.","errors":{"Titulo":["Informe o título."]}}
                        """);
            }
        }
    }
    return Task.CompletedTask;
}));
builder.Services.AddCors(o => o.AddPolicy("FrontendLocal", p =>
p.WithOrigins(builder.Configuration["Portal:FrontendOrigin"] ?? "http://localhost:5173")
    .AllowAnyHeader().AllowAnyMethod().AllowCredentials().WithExposedHeaders("X-Correlation-ID", "Location")));
builder.Services.AddRateLimiter(o =>
{
    o.RejectionStatusCode = 429;
    o.AddPolicy("diagnostico", context => RateLimitPartition.GetFixedWindowLimiter(
        context.Connection.RemoteIpAddress?.ToString() ?? "local", _ => new FixedWindowRateLimiterOptions
        { PermitLimit = 5, Window = TimeSpan.FromSeconds(10), QueueLimit = 0, AutoReplenishment = true }));
    o.AddPolicy("api", context => RateLimitPartition.GetFixedWindowLimiter(
        context.Connection.RemoteIpAddress?.ToString() ?? "local", _ => new FixedWindowRateLimiterOptions
        { PermitLimit = 120, Window = TimeSpan.FromMinutes(1), QueueLimit = 0, AutoReplenishment = true }));
    o.OnRejected = async (context, ct) =>
    {
        context.HttpContext.Response.Headers.RetryAfter = "10";
        await Results.Problem(statusCode: 429, title: "Limite de requisições atingido. Tente novamente em
instantes.").ExecuteAsync(context.HttpContext);
    };
});
builder.Services.AddOutputCache(o => o.AddPolicy("Publico", p => p.Expire(TimeSpan.FromSeconds(15)).Tag("painel-
publico")));
builder.Services.AddHealthChecks().AddCheck<BancoHealthCheck>("banco", tags: ["ready"])
    .AddCheck<DependenciaHealthCheck>("dependencia", tags: ["ready"]);
builder.Services.AddHttpClient("dependencia", c => c.Timeout = TimeSpan.FromSeconds(15))
```

```
    .AddStandardResilienceHandler(o =>
    {
        o.Retry.MaxRetryAttempts = 2; o.Retry.Delay = TimeSpan.FromMilliseconds(200); o.Retry.UseJitter = true;
        o.Retry.DisableForUnsafeHttpMethods();
        o.AttemptTimeout.Timeout = TimeSpan.FromSeconds(3);
        o.TotalRequestTimeout.Timeout = TimeSpan.FromSeconds(12);
        o.CircuitBreaker.SamplingDuration = TimeSpan.FromSeconds(30);
        o.CircuitBreaker.MinimumThroughput = 4; o.CircuitBreaker.FailureRatio = .5;
        o.CircuitBreaker.BreakDuration = TimeSpan.FromSeconds(10);
    });
var telemetry = builder.Services.AddOpenTelemetry().ConfigureResource(r => r.AddService(Telemetria.Nome));
telemetry.WithTracing(t =>
{

t.AddAspNetCoreInstrumentation().AddHttpClientInstrumentation().AddSqlClientInstrumentation().AddSource(Telemetria.Nome);
    if (builder.Configuration.GetValue<bool>("Telemetry:Console")) t.AddConsoleExporter();
});
telemetry.WithMetrics(m =>
{
    m.AddAspNetCoreInstrumentation().AddHttpClientInstrumentation().AddMeter(Telemetria.Nome);
    if (builder.Configuration.GetValue<bool>("Telemetry:Console")) m.AddConsoleExporter();
});

var app = builder.Build();
var memory = app.Configuration["Portal:Storage"] == "Memory";
if (!memory && string.IsNullOrWhiteSpace(app.Configuration.GetConnectionString("PortalTarefas")))
    throw new InvalidOperationException("Configure ConnectionStrings:PortalTarefas por User Secrets ou variável de
ambiente.");
if (memory && !app.Environment.IsDevelopment() && !app.Environment.IsEnvironment("Testing"))
    throw new InvalidOperationException("O armazenamento Memory é permitido somente para demonstração em Development ou
Testing.");
if (args.Contains("--initialize"))
{
    if (memory) throw new InvalidOperationException("A inicialização de banco exige SqlServer.");
    await using var scope = app.Services.CreateAsyncScope();
    await scope.ServiceProvider.GetRequiredService<PortalTarefasDbContext>().Database.MigrateAsync();
    await SeedData.InicializarAsync(app.Services, app.Configuration);
    return;
}
if (memory && builder.Configuration.GetValue<bool>("Seed:SampleData"))
app.Services.GetRequiredService<InMemoryTarefaService>().PopularExemplos();
if (!memory && builder.Configuration.GetValue<bool>("Seed:Enabled")) await SeedData.InicializarAsync(app.Services,
app.Configuration);

app.UseRegistroRequisicoes();
app.UseExceptionHandler();
app.UseStatusCodePages(async context => await Results.Problem(statusCode:
context.HttpContext.Response.StatusCode).ExecuteAsync(context.HttpContext));
if (!app.Environment.IsDevelopment() && !app.Environment.IsEnvironment("Testing")) app.UseHsts();
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseCors("FrontendLocal");
app.UseAuthentication();
app.UseAuthorization();
app.UseRateLimiter();
app.UseOutputCache();
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
    app.UseSwaggerUI(o => { o.SwaggerEndpoint("/openapi/v1.json", "Portal de Tarefas v1"); o.RoutePrefix = "swagger"; });
}
app.MapHealthChecks("/health/live", new HealthCheckOptions { Predicate = _ => false });
app.MapHealthChecks("/health/ready", new HealthCheckOptions { Predicate = check => check.Tags.Contains("ready") });
app.MapGet("/version", () => new VersionDto(typeof(Program).Assembly
    .GetCustomAttributes(typeof(System.Reflection.AssemblyInformationalVersionAttribute), false)
    .Cast<System.Reflection.AssemblyInformationalVersionAttribute>().FirstOrDefault()?.InformationalVersion ?? "local"));
app.MapGet("/api/minimal/diagnostico/{id:int}", (int id, string? filtro,
    [FromHeader(Name = "X-Cliente")] string? cliente, HttpRequest request) => new DiagnosticoDto(id, filtro, cliente,
```

```
request.Method))
    .WithName("DiagnosticoMinimal").WithSummary("Binding de rota, query e cabeçalho em Minimal
API.").RequireRateLimiting("diagnostico");
app.MapGet("/api/publico/painel", async (ITarefaService service, HttpContext context, CancellationToken ct) =>
{
    // Usa SEMPRE o escopo anônimo: cache nunca contém estatística privada.
    context.Response.Headers["X-Cache-Gerado-Em"] = DateTimeOffset.UtcNow.ToString("O");
    return await service.PainelAsync(new UsuarioContexto(null, "Geral", false, false, false), ct);
}).CacheOutput("Publico").WithName("PainelPublico");
app.MapControllers();
app.MapControllerRoute("default", "{controller=Tarefas}/{action=Index}/{id?}");
app.MapFallbackToFile("app/{*path:nonfile}", "app/index.html");
await app.RunAsync();

public partial class Program { }
```

src/PortalTarefas.Web/Middleware/RequisicaoMiddleware.cs

```
using System.Diagnostics;
using System.Text.RegularExpressions;
using PortalTarefas.Web.Infrastructure;

namespace PortalTarefas.Web.Middleware;

public sealed partial class RequisicaoMiddleware(RequestDelegate next, ILogger<RequisicaoMiddleware> logger)
{
    [GeneratedRegex("^[a-zA-Z0-9_-]{1,64}$")]
    private static partial Regex CorrelacaoValida();

    public async Task InvokeAsync(HttpContext context)
    {
        var recebida = context.Request.Headers["X-Correlation-ID"].ToString();
        var id = CorrelacaoValida().IsMatch(recebida) ? recebida : Guid.NewGuid().ToString("N");
        context.TraceIdentifier = id;
        context.Response.Headers["X-Correlation-ID"] = id;
        context.Response.OnStarting(() =>
        {
            // O tratamento de exceção pode limpar os cabeçalhos antes de gerar Problem Details.
            context.Response.Headers["X-Correlation-ID"] = id;
            return Task.CompletedTask;
        });
        using var scope = logger.BeginScope(new Dictionary<string, object> { ["CorrelationId"] = id });
        var inicio = Stopwatch.GetTimestamp();
        logger.LogDebug("Pipeline entrada {Metodo} {Caminho}", context.Request.Method, context.Request.Path);
        try { await next(context); }
        finally
        {
            var duracao = Stopwatch.GetElapsedTime(inicio).TotalMilliseconds;
            Telemetria.DuracaoHttp.Record(duracao, new KeyValuePair<string, object?>("metodo",
MetodoMetrica(context.Request.Method)),
                new KeyValuePair<string, object?>("status", context.Response.StatusCode));
            logger.LogInformation("Pipeline saída {Metodo} {Caminho} {Status} {DuracaoMs}ms",
                context.Request.Method, context.Request.Path, context.Response.StatusCode, duracao);
        }
    }

    private static string MetodoMetrica(string metodo) => metodo switch
    {
        "GET" or "POST" or "PUT" or "DELETE" or "PATCH" or "HEAD" or "OPTIONS" or "TRACE" or "CONNECT" => metodo,
        _ => "_OTHER"
    };
}

public static class RequisicaoMiddlewareExtensions
{
```

```
    public static IApplicationBuilder UseRegistroRequisicoes(this IApplicationBuilder app) =>
app.UseMiddleware<RequisicaoMiddleware>();
}
```

src/PortalTarefas.Web/Controllers/DiagnosticoController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.RateLimiting;

namespace PortalTarefas.Web.Controllers;

public sealed record DiagnosticoDto(int Id, string? Filtro, string? Cliente, string Metodo);

[ApiController, Route("api/diagnostico"), AllowAnonymous, EnableRateLimiting("diagnostico")]
public sealed class DiagnosticoController : ControllerBase
{
    /// <summary>Demonstra binding de rota, query e cabeçalho no controller MVC.</summary>
    [HttpGet("{id:int}"), ProducesResponseType<DiagnosticoDto>(200)]
    public DiagnosticoDto Obter([FromRoute] int id, [FromQuery] string? filtro, [FromHeader(Name = "X-Cliente")] string?
cliente)
        => new(id, filtro, cliente, Request.Method);
}
```